SASTRA

ENGINEERING · MANAGEMENT · LAW · SCIENCES · HUMANITIES · EDUCATION

DEEMED TO BE UNIVERSITY

(U/S 3 of the UGC Act, 1956)

THINK MERIT | THINK TRANSPARENCY | THINK SASTRA

OPERATING SYSTEMS LABORATORY

Course Code: CSE309R01

Semester: V

Lab Manual

2026

SCHOOL OF COMPUTING

SHANMUGHA ARTS, SCIENCE, TECHNOLOGY AND RESEARCH ACADEMY

(SASTRA Deemed to be University)

Tirumalaisamudram, Thanjavur-613 401

Course Objective:

This course will help the learner to gain pragmatic knowledge on the different modules of operating system by simulating them and analysing their performance differences.

Course Learning Outcomes:

Upon successful completion of this course, the learner will be able to

- Develop a program to create parent/child or concurrent process and establish communication between them using pipes, shared memory and message queues
- Analyze CPU and thread scheduling algorithms by simulating them and providing the CPU schedule with sample test data and calculate performance related metrics
- Implement mutual exclusion based on semaphores to prevent concurrent issues and ensure process synchronization under classic problems of concurrency
- Demonstrate deadlock avoidance and detection strategies
- Simulate memory management schemes and disk scheduling schemes

List of Experiments:

1. Program to create of a child process using fork, trace the process ids and study about orphan, zombie and daemon process and create multiple children and establish communication between siblings and parent using pipes
2. Program to implement IPC using Shared Memory System Call
3. Program to implement IPC with the help of Message Queues.
4. Programs for the simulation of Non pre-emptive scheduling algorithms and with CPU, I/O Burst time and analyze their performances.
5. Programs for the simulation of pre-emptive scheduling algorithms and analyze their performances
6. Program to experiment multi-level feedback scheduling.
7. Program for the simulation of thread scheduling approaches and implement Peterson's solution for enforcing mutual exclusion.
8. Program to demonstrate process synchronization for producer-consumer problem.
9. Program to demonstrate process synchronization for reader-writer problem.
10. Program to demonstrate process synchronization for dining philosopher's problem.
11. Program to Implement banker's algorithm for deadlock handling.
12. Program to implement First Fit, Best Fit, and Worst Fit strategies to allocate memory to a set of processes.
13. Program to simulate page replacement algorithms and calculate the number of page faults.
14. Program to implement disk scheduling algorithms.

Additional Experiments:

1. Program to simulate dynamic partitioning and buddy system.
2. Program to demonstrate file allocation techniques.

Exercise No. 1. Parent-Child process creation

i. Creation of a child process and communication between parent and children

Objectives

To create a child process using fork system call and use pipe for interaction between parent and child

Pre-requisite:

Knowledge of parent-child process, fork and pipe commands.

Theory or Concept

Fork () - creates a new process by duplicating the calling process. The new process is referred to as the child process. The calling process is referred to as the parent process.

Syntax: pid_t fork(void);

Return Values:

- On success, the PID of the child process is returned in the parent, and 0 is returned in the child. On failure, -1 is returned in the parent, no child process is created, and errno is set to indicate the error
-

Pipe () - creates a pipe, a unidirectional data channel that can be used for interprocess communication. The array pipe_fd is used to return two file descriptors referring to the ends of the pipe. pipefd[0] refers to the read end of the pipe. pipefd[1] refers to the write end of the pipe.

Procedure / Algorithm

- Develop the parent process with code for calls to fork and pipe
- Child process created as a result of fork ()
- Write a message into pipe under parent part of the code
- Suspend parent process to invoke child
- Reads the message from the pipe under the child part of the code
- Parent and child terminates

Sample Input:

Output:

Observation

Write a program to create of a child process using fork, trace the process ids and study about orphan, zombie and daemon process

1. **Create a child process using fork().**
2. **Trace the process IDs using getpid() and getppid().**
3. **Investigate creation of orphan zombie and deamon process**

Write the syntax of the system calls to implement the above:

Observation

ii. **Write a program to create of a child process using fork, and establish communication between parent, child and multiple children using pipes**

1. **Create a child process using fork()**
2. **Demonstrates the usage of pipes for inter-process communication between a parent process and multiple child processes.**

Write the syntax of the system calls to implement the above:

Exercise No. 2 **IPC Using Shared Memory System Call**

Write program to carry out inter-process communication using shared memory

Objectives

To implement IPC using shared memory concept with the help of the library functions available.

Prerequisite

- Knowledge of IPC, Shared memory functions, their syntaxes and functionalities

Shared Memory:

- Shared memory is a memory shared between two or more processes. Each process has its own address space;

Procedure

❑Create the sender process and receiver process

❑Create a shared memory making using the appropriate function

❑Sender pushes its message into shared memory

❑Receiver retrieves the message and displays it to the user

Sample Input:

Output:

Observation

Summary of Shared Memory System Calls: Write the syntax of the following in detail:

1. **shmget(): Creates or accesses a shared memory segment.**
2. **shmat(): Attaches the shared memory segment to the calling process's address space.**
3. **shmdt(): Detaches the shared memory segment from the process's address space.**
4. **shmctl(): Controls and manages shared memory segments (e.g., removing, setting permissions, retrieving status).**

Exercise No. 3 **IPC using Message Queue**

Write program to carry out inter-process communication using message queue

Objectives

To implement IPC using message queues with the help of the library functions available

Prerequisite

Knowledge of IPC, Message queues, syntax and functionalities

Theory

- A **message queue** is an inter-process communication (IPC) mechanism that allows processes to exchange data in the form of messages between two processes
- It allows processes to communicate asynchronously by sending messages to each other where the messages are stored in a queue, waiting to be processed, and are deleted after being processed.

Procedure

❑Create the sender process and receiver process

❑Create a message queue using the appropriate functions

❑Sender pushes its message into message queue using msgsnd

❑Receiver retrieves the message using msgrcv and show it to the user

Sample Input

Output

Observation

Summary of Message Queue System Calls: Write the syntax of the following in detail:

1. **msgget()**: Creates or accesses a message queue.
2. **msgsnd()**: Sends a message to the message queue.
3. **msgrcv()**: Receives a message from the message queue.
4. **msgctl()**: Controls the message queue (e.g., retrieves status, removes the queue, changes permissions).

Exercise No. 4. Simulation of Non pre-emptive CPU Scheduling algorithms

Write programs to demonstrate pre-emptive scheduling algorithms and compare their performances

Objectives

Simulation of non pre-emptive CPU Scheduling algorithms with I/O and CPU burst times

Prerequisite

Knowledge of scheduling algorithms

Procedure

❑ Input the number of processes to be scheduled

❑ Input the arrival time and CPU burst time of each process

❑ Calculate the turnaround time and the waiting time of the processes based on the following scheduling methods:

- First Come First Serve (FCFS), Shortest Job First (SJF),

❑Compare the mean turnaround time and choose the algorithm providing the best result.

Sample Input

Output :(Gant chart to be displayed)

Execution Order:

Output performance measures

Throughput
Response time,
Turnaround time
Waiting time,
Average of all the above times

Observation

Workout a sample test case and estimate all times, Gantt chart for both FCFS and SJF scheduling policy with CPU and I/O Burst time

Exercise No. 5. **Simulation of CPU Scheduling with pre-emptive policies**

Write programs to demonstrate pre-emptive scheduling algorithms and compare their performances

Objectives

Simulation of CPU Scheduling algorithm with pre-emption – Round robin, SRTF

Prerequisite

Knowledge of scheduling algorithms

Procedure

❑ Input the number of processes to be scheduled

❑ Input the arrival time, CPU burst time1, IO burst time and CPU burst time2 of each process

❑ Calculate the turnaround time and the waiting time of the processes based on the FCFS

Sample Input

Output

Observation

Workout a sample test case and estimate all times, Gantt chart for both round robin and SRTF scheduling policy with appropriate time quantum for round robin

Exercise No. 6. Simulate multilevel Feedback Queue Scheduling

Write programs to demonstrate multi-level feedback queue scheduling

Objectives

Simulation of CPU Scheduling algorithm based on multi-level feedback queue

Prerequisite

Knowledge of scheduling algorithms

Procedure

❑ Input the number of processes to be scheduled

❑ Input the arrival time and CPU burst time of each process

❑Input the number of queues and their time quantum

❑ Implement the last queue as following FCFS

❑ Calculate the turnaround time and the waiting time of the processes

Sample Input

Output

Exercise No. 7. **Simulate Peterson's Algorithm**

Write a program to implement Peterson algorithm for synchronization

Objectives

To Simulate Peterson's algorithms for mutual exclusion for two process and then to extend to n process

Prerequisite / Tools Required (optional)

Knowledge of critical-sections, mutual exclusion, bounded waiting, Synchronization and producer-consumer problem

Procedure / Algorithm

- ❑ Create user level threads using pthreads
- ❑ Develop a function that access a shared resource through Peterson
- ❑ Associate that function to two threads
- ❑ Implement Peterson logic for mutual exclusion

Sample Input

Output

Observation

Note: Compilation command: gcc -o thread_program.c -pthread

Write syntax for pthread_create(), pthread_join(), and other thread-related functions from the POSIX Threads (pthread) library.

Extend Peterson solution for ‘n’ process from ‘2’ process. Write syntax of the system calls used to implement the solution.

Exercise No. 8. **Simulation of Producer-Consumer problem**

Write a program to simulate producer consumer problem

Objectives

Implementation of Producer-Consumer problem using bounded and unbounded variations

Prerequisite / Tools Required (optional):

Knowledge of Concurrency, Mutual exclusion, Synchronization and producer-consumer problem

Procedure / Algorithm

- ❑ Implement producer-consumer program with producers and consumers simulated as threads.
- ❑ Employ necessary semaphores for bounded and unbounded implementations
- ❑ Run the program to allow the producer and consumer share the buffer by synchronizing themselves through mutual exclusion

Sample Input

Output

Observation

Simulate producer consumer problem for ‘m’ producer and ‘n’ customer accessing a bounded buffer of limited size each in a separate window. Include the pseudo code for overflow and underflow conditions.

Exercise No.9. **Simulating Reader-Writer problem**

Write a program simulate reader-writer problem

Objectives

To write a code to solve the readers writer's problem based on reader priority and writer priority solution

Prerequisite / Tools Required (optional)

Knowledge of Concurrency, Mutual exclusion, Synchronization and Reader writer problem

Procedure / Algorithm

- ❑ Create a reader process
- ❑ Create a writer process
- ❑ Implement necessary semaphores
- ❑ Implement the programs giving reader priority and writer priority

Sample Input

Output

Observation

Write the pseudo code for 'm' readers and 'n' writers to be invoked in separate window and demonstrate the cases for 1) readers with priority and 2) writers with priority

Exercise No. 10: Dining Philosopher's problem

Write a program to simulate Dining Philosopher's problem

Objectives

To design a solution that ensures the philosophers can safely share the forks and avoid deadlock or starvation (goes hungry).

Theory or Concept

Each philosopher must alternate between thinking and eating. Eating is possible only when two forks are available. It is used to handle synchronization and concurrency in process execution related to resource sharing and mutual exclusion.

Procedure / Algorithm

Step1: Represent each chopstick with a semaphore.

Step2: A philosopher tries to grab a chopstick by executing a wait () operation on that semaphore.

Step3: A philosopher releases her chopsticks by executing the signal () operation on the appropriate semaphores.

When each philosopher tries to grab her right chopstick, philosopher will be delayed forever (deadlock) to handle the situation allow a philosopher to pick up her chopsticks only if both chopsticks are available (the philosopher must pick them up in a critical section).

Sample Input

Interleaving operation of eating and thinking will be displayed for different philosophers

The solution guarantees that no two neighbours are eating simultaneously

Output

Observation

Write the pseudo code for dining philosopher problem

Exercise No. 11: Implementation of Banker's Algorithm for Deadlock Avoidance

Write a program to simulate banker's algorithm for deadlock avoidance

Objectives

The primary goal of the Banker's Algorithm is to prevent deadlock while allocation of resources to processes, where multiple processes compete for limited resources (such as CPU time, memory, or devices).

Theory or Concept

The algorithm ensures that the system remains in a safe state during resource allocation.

A safe state allows all processes to complete their execution without getting stuck due to resource unavailability.

Procedure / Algorithm

Step 1: Input the values of Max, Allocation and Resources arrays

Step 2: Calculate values of Need and Available

Step 3: Implement safety algorithm to determine whether the system is in safe state or not

Step 4: Input additional resource request of process

Step 5: If request > Max then report error

Step 6: If request > Available then return resource unavailable

Step 7: Allocate resource as per request and invoke safety algorithm to determine whether the system is in safe state or not

Step 8: If safe state then requests accepted else request rejected

Sample Input / Output

	Allocation	*Max*	*Available*
	A B C	*A B C*	*A B C*
P_0	0 1 0	7 5 3	3 3 2
P_1	2 0 0	3 2 2	
P_2	3 0 2	9 0 2	
P_3	2 1 1	2 2 2	
P_4	0 0 2	4 3 3	

Request 1: P2 makes a request for 1 additional unit of C

Request 2: P0 requests one unit of A and C

Output:

Observation

Solve a problem using bankers' algorithm and check for the possibility of deadlock. If there is no deadlock generate the safe sequence.

Exercise No. 12 Implementation of Memory Allocation strategies (First Fit, Best Fit, Worst Fit)

Objectives:

To implement a C program to simulate the concept of memory allocation strategies. Memory allocation strategies play a significant role in deciding how memory blocks are assigned to running processes.

In this exercise three memory allocation strategies are explored

1. **First Fit**
2. **Best Fit**
3. **Worst Fit**

Each of these strategies allocates memory in a different way, aiming to optimize performance, minimize fragmentation, or balance both.

The **objective** of this lab is to:

- Implement and understand the basic working of these three memory allocation strategies.
- Simulate the process of memory allocation and deallocation.
- Observe how each algorithm handles memory fragmentation and efficiency.

Theory or Concept:

Algorithm for Each Strategy:

a. First Fit Algorithm:

1. Iterate through the list of free memory blocks.
2. Find the **first block** that is **large enough** to accommodate the process.
3. Allocate the memory from that block and update the free block list.
4. If no suitable block is found, the allocation fails.

b. Best Fit Algorithm:

1. Iterate through the list of free memory blocks.
2. Find the **smallest block** that can accommodate the process.
3. Allocate the memory from that block and update the free block list.
4. If no suitable block is found, the allocation fails.

c. Worst Fit Algorithm:

1. Iterate through the list of free memory blocks.
2. Find the **largest block** that can accommodate the process.
3. Allocate the memory from that block and update the free block list.
4. If no suitable block is found, the allocation fails

1. **Implement the memory allocation strategies in C or C++:**
   - Use a structure to represent a memory block with size, start address, and allocation status (free or allocated).
   - Implement functions for each of the three algorithms: first_fit(), best_fit(), and worst_fit().
2. **Simulate memory allocation and deallocation:**
   - Create a list of processes with varying memory size requests.
   - For each process, simulate memory allocation using all three strategies.
   - After each process completes, release the memory and simulate memory deallocation.
3. **Generate statistics:**
   - Track the number of allocations and failures.
   - Monitor memory fragmentation (external and internal).
   - Display memory usage and efficiency after each allocation and deallocation.

4. **Implement the memory allocation strategies** in C or C++:
   - Use a structure to represent a memory block with size, start address, and allocation status (free or allocated).
   - Implement functions for each of the three algorithms: first_fit(), best_fit (), and worst_fit ().
5. **Simulate memory allocation and deallocation**:
   - Create a list of processes with varying memory size requests.
   - For each process, simulate memory allocation using all three strategies.
   - After each process completes, release the memory and simulate memory deallocation.
6. **Generate statistics**:
   - Track the number of allocations and failures.
   - Monitor memory fragmentation (external and internal).
   - Display memory usage and efficiency after each allocation and deallocation.

Observation

For the given process request set for memory in terms of size, show the memory allocation pattern with internal and external fragment size for all three memory allocation strategies (First Fir, Best Fit and Worst Fit)

Exercise 13 Program to simulate page replacement algorithms and to compute number of page faults

Objectives:

To simulate page replacement algorithms.

Prerequisite/ Tools Required:

Knowledge of Paging concepts, replacement algorithms

Theory or Concept:

Page replacement algorithms are an important part of virtual memory management and it helps the OS to decide which memory page can be moved out making space for the currently needed page. However, the ultimate objective of all page replacement algorithms is to reduce the number of page faults.

FIFO-This is the simplest page replacement algorithm. In this algorithm, the operating system keeps track of all pages in the memory in a queue, the oldest page is in the front of the queue. When a page needs to be replaced page in the front of the queue is selected for removal.

LRU-In this algorithm page will be replaced which is least recently used

OPTIMAL- In this algorithm, pages are replaced which would not be used for the longest duration of time in the future. This algorithm will give us less page fault when compared to other page replacement algorithms.

Procedure /Algorithm:

Step 1: Input the number of pages and number of frames

Step 2: Input list of page numbers into an array a[i]

Step 3: Implement the page replacement algorithm FIFO/LRU/Optimal

Step 4: Calculate the number of page faults for the given page numbers

Step 5: Print the results.

Sample Input

2 -1 -1
2 3 -1
2 3 -1
2 3 1
5 3 1
3 2 4
3 2 4
3 5 4
3 5 2

Output

Number of page faults: 9, No. Of page hits: Hit ratio: Miss ratio:

Observation

Solve the problem for various page replacement policies for a given working set and number of frames and compare the hit ratio and miss ration for all the policies for the same problem.

Exercise No. 14. Program to implement disk scheduling algorithms.

Objectives:

To Simulate of disk scheduling techniques.

Prerequisite / Tools Required:

Knowledge of disk scheduling algorithms.

Theory/ Concept:

One of the responsibilities of the operating system is to use the hardware efficiently. For the disk drives, meeting this responsibility entails having fast access time and large disk bandwidth. Both the access time and the bandwidth can be improved by managing the order in which disk I/O requests are serviced which is called as disk scheduling.

Different algorithms: FCFS, SSTF, SCAN, C-SCAN, LOOK, C-LOOK

Procedure \ Algorithm:

Step 1: Let Request array represents an array storing indexes of tracks that have been requested in ascending order of their time of arrival. 'head' is the position of disk head.
Step 2: Let us one by one take the tracks in default order and calculate the absolute distance of the track from the head.
Step 3: Increment the total seek count with this distance.
Step 4: Currently serviced track position now becomes the new head position.
Step 5: Go to step 2 until all tracks in request array have not been serviced.

- Calculate the seek time as per the algorithms listed below:
- First-in-First-Out
- Shortest Seek Time First
- Scan
- C-look

Sample Input:

A disk has **200 tracks (0–199)**.
The disk request queue is: 98, 183, 37, 122, 14, 124, 65, 67

Calculate the **total head movement** using:

1. FCFS
2. SSTF
3. SCAN
4. C-look

Output / Performance Measures:

Observation

Solve the problem and calculate the seek time and average time for the given request set for all the disk scheduling policies and recommend the best algorithm for the given input.